what is this?

Is that a mushroom?

clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

REST API and X as a Service

Updates

NOT PL

1. MP 9 - weather filter due next week ON YOUR VM
    a. Fix released at — redownload the tests
2. HW 8 due next Wednesday
3. CA applications out, deadline April 27th

REST API and X as a Service

Today's LGs - Build up your mental model of the websites and APIs

Be able to identify if an API is following REST principles

Be able to tell the difference and similarities between a website and an API

Be able to take steps to host your own website

Be able to describe the three X as a Service types

Be able to name one consideration of cloud computing at scale

Agenda

1. REST APIs
2. X as a service
3. Distributed Computing

When you use a web API your program (XXXX) creates and sends an HTTP request.

A. Client

B. Server

clicker.cs.illinois.edu

Q2

~Code~

340

The request is split into packets and routed across the internet to the correct computer based on the XXXX.

clicker.cs.illinois.edu

Q3

~Code~
340

A. TCP header

B. UDP header

C. IP address

D. HTTP information

The packets arrive at the correct computer and are sent to the correct port which is found in the packet's XXXX layer.

clicker.cs.illinois.edu

Q4

~Code~
340

A. Internet Layer (IP)

B. Transport Layer (TCP)

C. Payload (HTTP)

The API (XXXX) is listening at that port and receives the request, processes the information, and responds with a HTTP response.

clicker.cs.illinois.edu

Q5

~Code~
340

A. Client

B. Server

Web API - a web interface that acts as a server

Server + documentation

waits for requests

Sends response

REST - set of principles to help streamline working with web APIs

REST Principles

Uniform Interface – HTTP

Stateless –

REST Uniform Interface - HTTP Requests

Methods as verbs (GET, PUT, POST, DELETE)

URL is the resource ← end point specifies the resource

good GET url/users/123

Bad ~~GET~~ POST url/getUser?id=123

REST Uniform Interface - HTTP Requests

Bad /getUser

```
@routes.get('/getUser')
async def get_user(request):
    user_id = request.rel_url.query['id']
    return web.json_response({"id": user_id, "name": "Alice"})
```

Good REST

```
@routes.get('/users/{id}')
async def get_user(request):
    user_id = request.match_info['id']
    return web.json_response({"id": user_id, "name": "Alice"})
```

REST Uniform Interface - HTTP Responses

2XX - Success

returns JSON or nothing

4XX - Client Error

5XX - Server Error

REST Uniform Interface - Stateless

Everything the server needs to respond to a request is in the HTTP request and not from a prev request

Why? spread out traffic across many servers
if a server crashes it's ok

Stateless Misnomer -

- store things in a database
  can remember things about the user
  can't remember a previous specific request

Was this RESTful?

NO

```
class Game:
    def __init__(self):
        self.word = "flower"
        self.lives = 5
        self.guesses = []
        self.status = ["-"] * 6

    def guess(self, letter):
        # logic of what to do with a guess
        return None

grm = Game()
@routes.get("/")
async def get_page(request: web.Request) -> web.Response:
    return web.Response(text=open('spring_game.html').read(), content_type='text/html; charset=utf-8')

@routes.post("/guess")
async def post_guess(request: web.Request) -> web.Response:
    letter = await request.text()
    result = grm.guess(letter)
    return web.Response(text=result)
```

clicker.cs.illinois.edu

Q6

~Code~
340

What could we change?

clicker.cs.illinois.edu

Q7

~Code~

340

```
class Game:
    def __init__(self):
        self.word = "flower"
        self.lives = 5
        self.guesses = []
        self.status = ["-"] * 6

    def guess(self, letter):
        # logic of what to do with a guess
        return None

grm = Game()
@routes.get("/")
async def get_page(request: web.Request) -> web.Response:
    return web.Response(text=open('spring_game.html').read(), content_type='text/html; charset=utf-8')

@routes.post("/guess")
async def post_guess(request: web.Request) -> web.Response:
    letter = await request.text()
    result = grm.guess(letter)
    return web.Response(text=result)
```

```python
@routes.post("/game")
async def create_game(request: web.Request) -> web.Response:
    game = {"word": "flower", "lives": 5, "guesses": [], "status": ["_"] * 6}
    game_id = save_game(game)  # save to database
    return web.json_response({"game_id": game_id}, status=201)

@routes.get("/game/{game_id}")
async def get_game(request: web.Request) -> web.Response:
    game_id = request.match_info['game_id']
    game = load_game(game_id)  # fetch from database
    return web.json_response(game)

@routes.post("/game/{game_id}/guess")
async def post_guess(request: web.Request) -> web.Response:
    game_id = request.match_info['game_id']
    game = load_game(game_id)  # fetch from database
    body = await request.json()
    letter = body.get("letter")
    # guess logic here
    save_game(game_id, game)  # save updated state to database
    return web.json_response(game)
```

```python
WORD = "flower"
@routes.post("/guess")
async def post_guess(request: web.Request) -> web.Response:
    body = await request.json()
    letter = body.get("letter")
    guesses = body.get("guesses", [])
    lives = body.get("lives", 5)
    status = body.get("status", ["-"] * 6)

    if letter in guesses:
        return web.json_response({"error": "already guessed"}, status=400)

    guesses.append(letter)
    if letter in WORD:
        for i, l in enumerate(WORD):
            if l == letter:
                status[i] = letter
    else:
        lives -= 1

    return web.json_response({"guesses": guesses, "lives": lives, "status": status})
```

I have 5 servers running my REST API. A client sends a request to server 1, where does the next request from that client need to go?

clicker.cs.illinois.edu

Q8

~Code~
340

A. Server 1

B. Server 2

C. Server 3 or 4 or 5

D. Any of the servers

Can I require clients to login before using my REST API? Given that it is stateless?

clicker.cs.illinois.edu

Q9

~Code~
340

A. Yes, with a database

B. Yes, with the server saving internal state

C. No

REST Principles - convention and consistency

Uniform Interface

Stateless is more common with server design

Web API

client - a program that will process the results

server - end points return JSON

↑

API

BOTH

use the internet to send information

usually use HTTP and client-server model

Website

client - a browser application

server - HTTP end points returns HTML + other stuff

Website Requirements

Domain name → public IP address ← stored in DNS servers

At the public IP address → computer
needs to be running a process
that is acting like a server ← listening on a well known port → 80 HTTP

GET / request with html

Hosting a Website

X personal laptop ← doesn't have public IP address

✓ get a computer built to be a server

- set it up

black out?

X as a Service

Pay someone to do the hard parts

? what is difficult?

Cloud – a buisness model for renting compute resources

AWS

IaaS
PaaS
SaaS

Infrastructure as a Service IaaS

- a virtual machine that lives somewhere else physically
  - you don't maintain the hardware
  - you do set it up and deal w/ scaling

AWS – rent a virtual machine come with a public IP address

access through SSH

Platform as a Service

virtual machine + other stuff

example - AWS provides databases

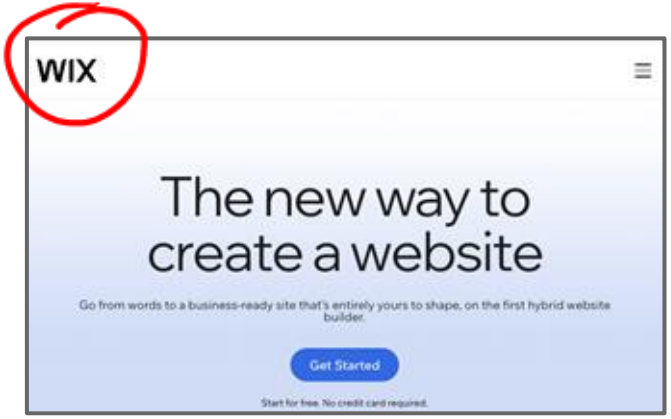
they handle the scaling

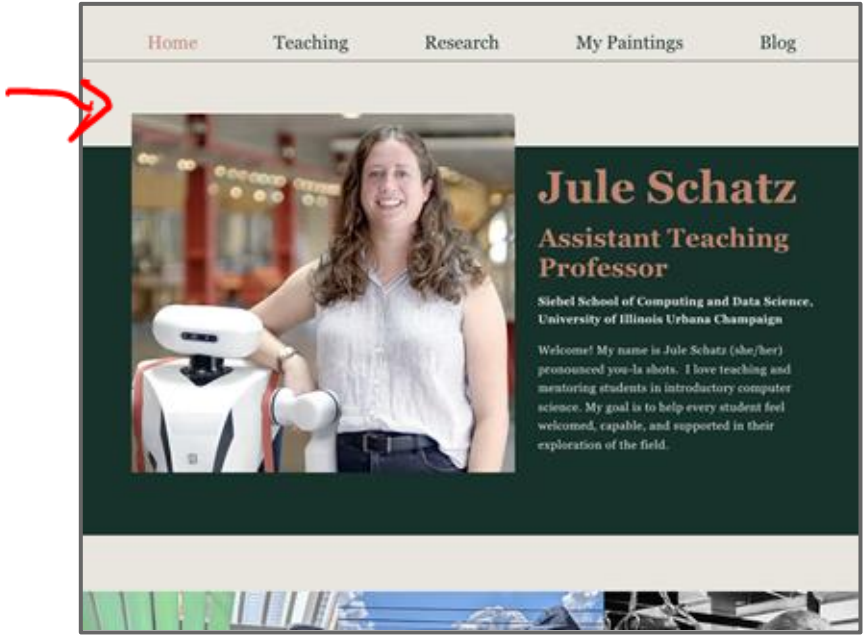
Software as a Service

SaaS

a user friendly UI

spotify





X as a Service Analogy

IaaS – give you a kitchen
– you buy ingredients and cook

PaaS – gives you a kitchen and the premade dough

SaaS – order pizza for delivery

Hosting a Website

need IP address, domain name, server running

public

A) your own hardware

B) IaaS - get a virtual machine

C) SaaS - use WIX

D) Host a static website

- services that host HTML and other files for you

Github pages, AWS

I want a website that is just an HTML file, to put it on the internet I can...

clicker.cs.illinois.edu

Q10

~Code~
340

A. Buy a domain name and set it up to grab the file from my personal laptop

B. Use a static hosting site that runs and creates the server for me that responds to GET requests with the html file

C. Get a virtual machine and store the html file on it. Then get a domain name and link it to the virtual machine.

you have to run the server

I want to build and deploy a REST API I can...

A. Get a domain name and then run the server code once on my computer to set it all up with an aiohttp API

B. Get a virtual machine and connect it with a domain name and then run the aiohttp code on the VM listening on port 80.

C. Use the WIX website to create and deploy the API

clicker.cs.illinois.edu

Q11

~Code~
340

How the Cloud Scales Servers

Old "build hardware that never fails"

99.9 % good

1 million servers → 1000 fails

New "assume hardware will fail, and design software around that"

Software Solution: Replication

everything is stored 3 times

Pros

- if one crashes we are good
- faster if it's closer to you

Cons

- More space - 3x
- Consistency - how to keep things synced